# Kubernetes Observability and Pod Design

Starts at 9:05 pm

## Agenda

1. Configmaps and Secrets
2. Resource Settings for pods in Kubernetes
3. Service Accounts
4. Pod Scheduling and Placement
5. Taints and Tolerations
6. Node Affinity/ Anti Affinity
7. Node Selectors and Labels

## Configmaps and Secrets

Configmap.

→ K8s object that is used to store Non confidential data in key-value pairs.

① Literal Values

key value value.

```
kubectl create configmap example-configmap --from-literal=configKey1=value1 --from-literal=configKey2=value2
```

key

② Configuration file

key → filename
value → file content

kubectl create configmap example-configmap --from-file=config-file.txt

③ From directories

kubectl create configmap example-configmap --from-file=config-directory/

→ Secrets K8s object
used to store sensitive information.
passwords, tokens, key.

base 64 encoded.

→ Opaque secret. (default)

```
apiVersion: v1
kind: Secret
metadata:
  name: my-opaque-secret
type: Opaque
data:
  username: dXNlcm5hbWU=  # base64 encoded value
  password: cGFzc3dvcmQ=  # base64 encoded value
```

→ Docker Registry Secret.

```
apiVersion: v1
kind: Secret
metadata:
  name: my-docker-secret
type: kubernetes.io/dockerconfigjson
data:
  .dockerconfigjson: base64-encoded-docker-config
```

```
kubectl create secret docker-registry my-registry-secret --docker-username=your-username --docker-password=your-
password --docker-email=your-email
```

TLS secret

```
apiVersion: v1
kind: Secret
metadata:
  name: my-tls-secret
type: kubernetes.io/tls
data:
  tls.crt: base64-encoded-tls-cert
  tls.key: base64-encoded-tls-key
```

Declarative way → Base64 encode
↓
manually.

Imperative way → K8s does encoding
for you.

Implement CM and Secret in Pod.

→ use as ENV variables.

Secrets

DB username → Secrets → K8s cluster.

DB password

Podspec.

key.

Pod

Cont.

ENV variable.

DB username & password.

my.cnf: >

[mysqld]

max_connections = 200

bind-address = 0.0.0.0

> → folded.

[mysqld] max_connections = 200 bind-address = 0.0.0.0 ??

CM → declaratively.
Secret → Imperatively.

CM → volume.
Secret → Env.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-config
data:
  my.cnf: |
    [mysqld]
    max_connections = 200
    bind-address = 0.0.0.0
```

```
kubectl create secret generic mysql-secret --from-literal=mysql-root-password=password123
```

```
apiVersion: v1
kind: Pod
metadata:
```

```
  name: mysql-pod
spec:
  containers:
  - name: mysql
    image: mysql:5.7
    env:
    - name: MYSQL_ROOT_PASSWORD
      valueFrom:
        secretKeyRef:
          name: mysql-secret
          key: mysql-root-password
    volumeMounts:
    - name: mysql-config-volume
      mountPath: /etc/mysql/my.cnf
      subPath: my.cnf
  volumes:
  - name: mysql-config-volume
    configMap:
      name: mysql-config
```

# Environment Variables.

```
apiVersion: v1

kind: Pod

metadata:

  name: env-vars-pod

spec:

  containers:

  - name: example-container

    image: busybox

    env:

    - name: MY_ENV_VAR

      value: "some_value"

    command: [ "sh", "-c", "echo $(MY_ENV_VAR)" ]
```

# Resource settings for Pods in Kubernetes.

**Resource Requests**:

- Resource requests are the minimum amount of CPU and memory that a container requires.

```
resources:

  requests:

    memory: "64Mi"
```

cpu: "250m" → 250 millicores.

**Resource Limits**:

- Resource limits are the maximum amount of CPU and memory that a container is allowed to use.

resources:

limits:

memory: "128Mi"

cpu: "500m"

Mi → Mebibytes

MB → Megabytes.

$1 \text{ Mib} = 2^{20} \text{ bytes} = 1048576 \text{ bytes}$

$1 \text{ MB} = 10^{6} \text{ bytes} = 1000000 \text{ bytes}.$

1 Mib = 1024 Kib.

1 Gib = 1024 Mib

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo-pod
spec:
  containers:
  - name: demo-container
    image: busybox
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
    command: ["sh", "-c", "while true; do echo 'Running...'; sleep 10; done"]
```

Importance of Resource Requests and Limits

1. Application performance.
2. Prevents resource starvation.
3. Optimizes Cluster Utilization.

Break → 10:30pm.

→ Eviction

① Soft → gracefully.
② Hard. → kill them.

Pod Priority
→ Priority Class → Cluster Scoped

```
apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:

  name: high-priority

value: 1000

globalDefault: false
```

→ true

```
description: "This priority class is for critical workloads."


---

apiVersion: scheduling.k8s.io/v1

kind: PriorityClass

metadata:
```

```
  name: low-priority

value: 100

globalDefault: false

description: "This priority class is for less critical workloads."
```

default → 0

globalDefault → true

> 1 billion (Reserved)

< 1 billion (Pods)

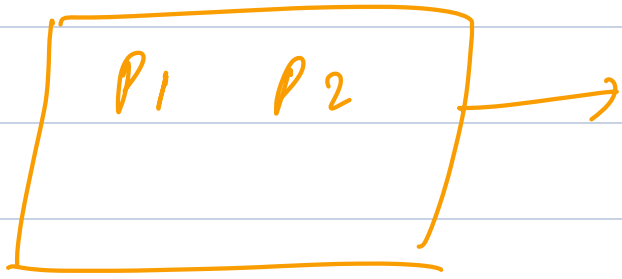
```
### High priority pod

apiVersion: v1

kind: Pod

metadata:

  name: high-priority-pod

  labels:

    app: critical-app

spec:
```

```
  priorityClassName: high-priority
  containers:
  - name: nginx
    image: nginx
```

### Low Priority Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: low-priority-pod
  labels:
    app: less-critical-app
spec:
  priorityClassName: low-priority
  containers:
  - name: busybox
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
```

→ Service Account.




user → cluster (API server)
↓
kubectl

Pod ?

**Service Accounts (SAs) are used by pods to authenticate with the Kubernetes API.**

Service Accounts provide an identity for processes that run in a Pod.

Service Accounts are used to provide access control and manage permissions for Pods.

Namespace specific

**Creating a service account**

kubectl create serviceaccount my-service-account

**yaml definition**

apiVersion: v1

kind: ServiceAccount

metadata:

  name: my-service-account

**Using a Service Account in a Pod**:

apiVersion: v1

kind: Pod

metadata:

RBAC

  name: pod-with-service-account

spec:

  serviceAccountName: my-service-account

  containers:

  - name: example-container

    image: busybox

    command: ["sh", "-c", "echo Hello Kubernetes!"]

# Using image PullSecrets with SA.

Secret → Docker Private Registry.

```
kubectl create secret docker-registry my-registry-secret --docker-username=your-username --docker-password=your-
password --docker-email=your-email
```

Secret → SA

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
imagePullSecrets:
- name: my-registry-secret
```

SA → Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-using-private-registry
```

```
spec:
  serviceAccountName: my-service-account
  containers:
  - name: example-container
    image: my-private-registry/my-image:latest
    command: ["sh", "-c", "echo Hello from private registry!"]
```

→ Create custom nginx.

```
kubectl patch serviceaccount my-service-account -p '{"imagePullSecrets": [{"name": "my-registry-secret"}]}'
```

→ existing.

## Pod scheduling and Pod Placement.

Pod scheduling is the process of assigning a Pod to a specific node in the Kubernetes cluster.

New Pod Created
↓
Sched detects it
↓
Filter Nodes.
↓
Feasible?

No / Yes

Pod scheduling Aborted

Scores Nodes

↓

Select Best Node

↓

Bind the pod to the node

↓

Pod scheduled.

- **Factors Influencing Pod Placement**:

  - **Resource Requests and Limits**:

    - Pods specify resource requests and limits for CPU and memory, which influence where they can be scheduled.

  - **Node Affinity/Anti-Affinity**:

    - These rules influence Pod placement by specifying whether a Pod should be placed near (affinity) or away from (anti-affinity) other Pods or nodes.

  - **Taints and Tolerations**:

    - Taints on nodes can repel certain Pods, while tolerations allow Pods to be scheduled on nodes with specific taints.

  - **Node Selectors and Labels**:

    - Node selectors and labels help in placing Pods on specific nodes that match certain criteria.

# Taints and Tolerations.

Taints → Rules → Nodes.
Toleration → Permissions → Pods.

→ Node (Husband) Taint
"Only my wife is allowed to stay with me"

→ Toleration → Pod
"I am the wife, so I can stay"
→ Components of taint
1. Key
2. Value
3. Effect.

```
kubectl taint nodes node1 key=value:NoSchedule
```
→ Node.

Different Effects.

1. NoSchedule

② PreferNoSchedule. (only to decrease node score)

role = frontend : PreferNoSchedule.

③ NoExecute.

Already running pods will be evicted.

role

frontend

NoExecute.

```
tolerations:
- key: "role"
  operator: "Equal"
  value: "frontend"
  effect: "NoSchedule"
```

```
kubectl taint nodes new-cluster1-worker role=frontend:NoSchedule
```

```
kubectl taint nodes new-cluster1-worker2 role=frontend:NoSchedule
```

```
apiVersion: v1
kind: Pod
metadata:
  name: toleration-pod
spec:
  containers:
  - name: example-container
    image: nginx
  tolerations:
  - key: "role"
  operator: "Equal"
    value: "frontend"
    effect: "NoSchedule"
```